BASHIR AGRO

Flutter Mobile Prototype Specification

Owner + Farmer Android App | No API Prototype

Prototype Scope

This document describes exactly how the Bashir Agro Flutter prototype should behave before Laravel/API integration. All users, farms, locations, tasks, submissions, incidents, notifications and payroll records are represented by local mock data.

Prepared as a development brief for Flutter implementation and AI coding agents.

1. Product Overview

Bashir Agro is an Android Flutter application with two mobile roles: Owner and Farmer. Admin is intentionally excluded from the mobile app because Bashir Agro staff will later use a separate Laravel web application. The prototype should reproduce the full business workflow locally so that navigation, role logic, farm/location management, task assignment, GPS validation, live photo proof, submissions, approval/rejection, incidents and reporting can be tested before any backend exists.

> **Core Prototype Principle**
>
> The app must feel like a real multi-user connected system, but every operation is performed against local mock repositories and local state. No REST API, database server, Firebase backend, SMS service or cloud storage is required at this stage.

2. Prototype Architecture

```
Flutter App
│
├── Local Mock Data
├── Local State Management
├── Local Authentication Simulation
├── Device Camera
├── Device GPS
├── Local Geofence Calculation
├── Local Task State Updates
└── Local Approval / Rejection Simulation
```

Recommended prototype stack:

- Flutter
- Riverpod for state management
- go_router for navigation
- camera for proof capture
- geolocator for GPS
- permission_handler for runtime permissions
- Hive (preferred) or SharedPreferences for persistence
- intl for date/time formatting
- url_launcher for phone calls
- google_maps_flutter when real map interaction is needed

Do not integrate the following during the prototype phase:

- Laravel REST API
- PostgreSQL/PostGIS server
- Firebase backend or push infrastructure
- SMS OTP
- Cloud/S3 image upload
- Server-side scheduling
- Real payroll transactions
- PDF/Excel report generation

3. Roles and Access Model

Role	Mobile Access	Main Responsibility
Farmer	Yes	View assigned work, verify location, capture live proof, submit tasks, report incidents, view notifications/profile/payroll.
Owner	Yes	Manage own farms/locations, assign existing Farmers, create tasks, review submissions, monitor incidents and reports.
Admin	No mobile app	Later managed through Laravel web; creates master Owner/Farmer accounts and has system-wide visibility.

Important

The prototype must demonstrate that one Farmer can work for multiple Owners at the same time, while one individual task is assigned to exactly one Farmer.

4. Local Mock Data and Repository Design

All screens should consume data through repository interfaces rather than reading hardcoded values directly from widgets. This ensures the mock repositories can later be replaced by API repositories without rewriting the UI or core workflow.

```
Screen
  ↓
Controller / Riverpod Notifier
  ↓
Repository Interface
  ↓
Mock Repository
  ↓
Local Mock Data / Hive
```

Suggested local data files during early development:

```
lib/data/
  mock_users.dart
  mock_farms.dart
  mock_locations.dart
  mock_tasks.dart
  mock_assignments.dart
  mock_submissions.dart
  mock_notifications.dart
  mock_incidents.dart
  mock_payroll.dart
```

Minimum core entities:

Entity	Purpose
User	Shared account data, role, phone, prototype password and

Entity	Purpose
	account status.
OwnerProfile / FarmerProfile	Role-specific profile data.
OwnerFarmerAssignment	Links an existing global Farmer to one or more Owners.
Property / FarmLocation	Owner property and operational field/pond/shed locations.
LocationAssignment	Authorizes a Farmer to work at a location, permanently or temporarily.
Task / TaskSchedule	One-time or recurring work allocated to one Farmer.
TaskPhoto	Local image path plus capture time, GPS and geofence result.
Submission / Review	Farmer evidence and Owner decision history.
Incident	Independent field problem report.
Notification	Local simulated notification feed.
PayrollRecord	Prototype Farmer monthly salary display.

5. Prototype Login and Role Routing

The app starts with the shared login screen. Authentication is simulated by matching the entered phone and password against the local user list. There is no remote authentication request.

Demo Account	Phone	Password	Expected Route
Farmer - Rahim Uddin	01700000001	123456	Farmer Dashboard
Owner - Abdul Karim	01700000002	123456	Owner Dashboard

```
if (user.role == 'farmer') {
  goToFarmerDashboard();
} else if (user.role == 'owner') {
  goToOwnerDashboard();
}
```

The user model should at least contain: id, name, phone, password (prototype only), role, profile photo and account status.

6. Farmer Experience - Overall Flow

```
Login
  ↓
Today's Work
  ↓
Task Details
  ↓
Start Task
  ↓
Location Check
  ↓
Perform Work
  ↓
Take Live Photos
  ↓
Photo Validation
  ↓
Submit
```

```
  ↓
Pending Owner Review
  ↓
Approved / Rejected
```

The Farmer UI should be intentionally simple. The Farmer does not manually switch Owners. All current tasks from every Owner are merged into a single daily work list.

7. Farmer Dashboard - “আজকের কাজ”

The Farmer dashboard is the main operational screen. It displays the current date, greeting, a compact daily summary and chronological task cards.

Card Field	Example
Task Title	মাছের খাবার দেওয়া
Owner	আব্দুল করিম
Farm / Property	Karim Agro Farm
Location	East Pond
Time	08:00 - 09:00
Status	চলমান / বাকি / রিভিউ অপেক্ষায় / সম্পন্ন
Flags	Late or location warning shown separately when applicable

Multi-Owner Rule

Rahim may have tasks from Abdul Karim, Bashir Ahmed and another Owner on the same day. All appear together. Every task card clearly displays its Owner and location.

8. Farmer Task States

```
scheduled
  ↓
started
  ↓
in_progress
  ↓
submitted
  ↓
pending_review
  ↓
approved / rejected
  ↓
completed / resubmission
```

Late is a separate boolean flag, not a separate primary task state. For example, a submission can be pending_review and isLate = true at the same time.

9. Task Details Screen

When the Farmer opens a task, show enough information to understand exactly what must be done before work begins.

- Task name
- Owner name
- Property/Farm name
- Assigned location
- Date
- Scheduled start
- Deadline
- Description
- Detailed instructions
- Minimum and maximum photo count
- Live-camera requirement
- Geofence/location verification requirement

The primary action is “কাজ শুরু করুন”.

10. Starting a Task and GPS Verification

When the Farmer presses Start Work, request location permission and read the current device location. Compare the GPS point to the mock polygon stored for that assigned location. The calculation can be entirely local in the prototype.

```
FarmLocation {
  id: "LOC001",
  name: "East Pond",
  polygonPoints: [...],
  toleranceMeters: 20
}
```

If the Farmer is inside the polygon or permitted tolerance, show a green confirmation and enable “কাজ শুরু নিশ্চিত করুন”. Record actualStartTime locally and update the task state to started.

Outside Location

If the Farmer is outside the valid area, show a red warning, map/polygon and current position. Do not allow a valid task start until the location check passes.

The app should not continuously track the Farmer. Location is checked at meaningful events: task start, photo capture and incident/report capture.

11. Prototype GPS Simulation Mode

Because demonstrations may happen from an office rather than a real farm, include a prototype-only simulation setting:

- Use actual GPS
- Simulate inside geofence
- Simulate outside geofence

Why this matters

The demo mode allows the entire valid/invalid geofence workflow to be presented without physically visiting the configured field or pond. Remove this control from production builds.

12. Live Camera Workflow

After a task starts, the Farmer taps “ছবি তুলুন”. Open the device camera directly using the camera package. Do not provide Gallery as a proof source.

- Show current task/location context
- Show current geofence status in green or red
- Show a photo counter such as 1/5
- Allow the shutter even when location is invalid
- Capture GPS again for every photo

```
TaskPhoto {
  id
  localPath
  capturedAt
  latitude
  longitude
  gpsAccuracy
  insideGeofence
}
```

Critical Rule

Every photo has its own independent GPS, capture time and geofence result. Do not store one coordinate for the entire submission.

13. Valid vs Invalid Photo Behavior

If the Farmer is inside the permitted location when taking a photo, mark the photo valid. If the Farmer is outside, still allow the photo to be captured but mark insideGeofence = false. Invalid photos remain visible for review but cannot count toward the required valid-photo total.

Example	Result
Photo 1 - inside field	Valid and counts toward minimum
Photo 2 - inside field	Valid and counts toward minimum
Photo 3 - outside field	Invalid; must be replaced for submission requirements

A watermark-style overlay can be shown in the prototype UI containing Bashir Agro, Farmer ID, capture time, location and coordinates. The prototype does not need to permanently burn this into the image file.

14. Photo Review Screen

After capture, show the photo, capture time, location, GPS coordinate, accuracy and verification result. Provide “আরেকটি ছবি তুলুন” and “ছবি রাখুন”.

If the photo is valid, show a green “অবস্থান সঠিক”. If invalid, show a red “নির্ধারিত মাঠের বাইরে তোলা হয়েছে” and clearly state that it cannot be used as valid proof.

15. Multiple Photos and Submission Eligibility

Each task defines minPhotos and maxPhotos. A task may require, for example, minimum 2 and maximum 5 photos. Only valid geofence photos count toward the minimum. The final Submit action remains disabled until all required conditions are met.

```
Valid photos: 2
Invalid photos: 1
Required minimum: 2
→ Submission enabled (invalid image does not count)
```

16. Farmer Task Submission

Before accepting a local submission, validate that the task belongs to the active Farmer, the task has been started, minimum valid photo count has been reached, maximum count is not exceeded, and every counted proof photo is valid.

Determine lateness locally by comparing current time with task.deadline. Late work is accepted but clearly flagged.

```
isLate = DateTime.now().isAfter(task.deadline);
```

Create a local Submission record, set task.status = pending_review, and show a success screen with Task ID, submission time, on-time/late status, number of photos and a 24-hour review due message.

17. Rejection and Resubmission

If the Owner rejects a submission, the Farmer receives a local notification and the task opens with the rejection reason. The Farmer can capture new live photos and submit again.

> **Audit Rule**
>
> Never overwrite the previous submission. Keep Submission #1 as rejected and create Submission #2 as a new record. This mirrors the future backend audit model.

18. Farmer History, Locations, Incidents and Profile

Farmer History should include filters such as সব, সম্পন্ন, দেরি and প্রত্যাখ্যাত. The Assigned Locations screen shows every currently permitted location with Owner, assignment period and map/polygon preview.

Incident reporting is independent of tasks. The Farmer selects a problem type, location, enters a description, takes a live photo and stores GPS. The local Incident then appears in the related Owner interface.

The Farmer profile shows name, Farmer ID, phone, address, emergency contact, employment status and assigned Owner count. A simple payroll section can show the monthly Bashir Agro salary record and paid/pending state.

19. Owner Experience - Overall Flow

```
Login
  ↓
Owner Dashboard
  ├── Farmers
```

```
    ├── Farms / Locations
    ├── Tasks
    ├── Approvals
    ├── Incidents
    ├── Reports
    ├── Notifications
    └── Profile
```

The Owner interface is more management-focused than the Farmer interface. It only exposes data belonging to that Owner and the Farmers linked to that Owner.

20. Owner Dashboard

The dashboard summarizes the Owner's current operation. Suggested cards: Assigned Farmers, Today's Tasks, Completed, Remaining, Pending Review and Overdue. A Farmer progress list can show percentages derived from mock tasks.

Metric	Example
Assigned Farmers	12
Today's Tasks	18
Completed	12
Remaining	6
Pending Review	5
Overdue	2

21. Owner Farmer List and Farmer Detail

The Owner sees only Farmers linked to that Owner. The list can show photo, name, phone, location count and status. Opening a Farmer shows basic contact details, emergency contact, assigned locations, today's tasks and recent activity.

> **Privacy Rule**
>
> Owner must not see Farmer NID, central salary or payroll details. Owner may see the Farmer's emergency contact and may call through the device dialer.

22. Owner Creates Farm Locations

The Owner can create a farm location directly in the prototype. Fields include location name, type, parent property, address, optional area and a polygon boundary.

- Vegetable Field
- Rice/Crop Field
- Fish Pond
- Cow Shed
- Poultry
- Orchard
- Greenhouse
- Storage

- Other

The location becomes active immediately after saving. No Admin approval is required.

23. Drawing and Storing a Geofence Polygon

A realistic prototype can use Google Maps to place multiple points and close a polygon. A simpler prototype may use a static map image or custom canvas. Either way, save a list of polygon points locally.

```
Location {
  id
  ownerId
  propertyId
  name
  type
  polygonPoints
  toleranceMeters = 20
  status = active
}
```

The 20m tolerance is a default. The value should be stored per location rather than hard-coded into the geofence logic.

24. Owner Assigns Existing Farmers

Owners do not create new Farmer accounts. In the prototype, assume a global list of Farmers already exists. The Owner selects an existing Farmer, chooses one or more Owner-owned locations, and optionally sets a start/end assignment period.

Assignment Type	Example
Permanent	Rahim → East Pond
Temporary	Rahim → North Field, 01 Oct - 15 Oct

The same Farmer may simultaneously be assigned to another Owner. This is a core scenario that must exist in the demo dataset.

25. Owner Task Management

The Owner Task area should provide Today, Upcoming, Recurring and History views. Each card contains task name, Farmer, location, time and status.

Creating a task requires: task name, Farmer, property, location, date, start time, deadline, instructions, minimum photos, maximum photos and recurrence.

Assignment Rule

One task is assigned to exactly one Farmer. A Farmer can have many tasks, but multiple Farmers cannot share one task record.

26. One-Time and Recurring Tasks

A one-time task creates one local task instance. A recurring task stores a recurrence type such as daily, weekly or monthly. The prototype does not need a full production scheduler; it can pre-generate enough upcoming task instances for demonstration.

```
recurrenceType = daily
Generate demo instances:
25 Sep
26 Sep
27 Sep
```

Future recurring tasks should not automatically switch to a new Farmer when a Farmer assignment changes. Reassignment remains manual, matching the agreed business rule.

27. Owner Approval Inbox

Pending Farmer submissions appear in an approval inbox. Each row shows Farmer, task, location, submission age, late/on-time indicator and review state.

The review detail screen shows the actual locally captured images plus capture time, coordinates, GPS accuracy and geofence verification for each photo. The Owner then chooses Approve or Reject.

28. Approval, Rejection and 24-Hour Review Window

Approving sets submission.reviewStatus = approved and task.status = completed, then creates a local Farmer notification. Rejecting requires a reason and optional comment, sets the submission to rejected and task to rejected, and creates a resubmission notification for the Farmer.

```
reviewDueAt = submittedAt.add(Duration(hours: 24));
```

If the local current time passes reviewDueAt, display Review Overdue. No server background job is required for the prototype.

29. Owner Incident Management

The Owner sees incidents reported by Farmers for the Owner’s locations. States: OPEN, ACKNOWLEDGED, IN PROGRESS and RESOLVED. The Owner can open the incident photo, location and description and update status locally.

30. Owner Reports and Notifications

Prototype reports should be calculated from mock tasks and submissions rather than stored as separate static report data. Show Farmer performance and location-level task totals. Export buttons may simply display a “production feature” message.

Owner notifications can include new submission, outside-geofence attempt, overdue task, new incident and review deadline warnings.

31. Local Notifications and Multi-User Simulation

Whenever one local action would normally cause a server notification, create a local Notification record for the other role. This makes the prototype behave like a connected system.

```
Farmer submits task
→ create Submission
→ create Owner notification

Owner approves
→ update Task / Submission
→ create Farmer notification
```

32. Demo User Switching

Add a development-only “Switch Demo User” option. This is extremely useful for end-to-end demonstrations without a backend.

```
1. Login as Rahim (Farmer)
2. Submit task
3. Switch to Abdul Karim (Owner)
4. Review and approve
5. Switch back to Rahim
6. See approval notification and completed state
```

33. Local Persistence and Reset Demo Data

For a useful prototype, user actions should survive app restarts. Hive is recommended for storing mutable local records such as tasks, submissions, photos metadata, assignments, incidents and notifications.

Provide “Reset Demo Data” in developer/settings. This clears changes and restores the original seed dataset so demonstrations can be repeated from a known state.

34. Recommended Seed Dataset

```
OWNER 1: Abdul Karim
Locations: East Pond, South Field, Cow Shed
Farmers: Rahim, Kamal

OWNER 2: Bashir Ahmed
Locations: North Field, Vegetable Block
Farmers: Rahim, Selim

Rahim therefore works for both Owner 1 and Owner 2.
```

Example Rahim tasks:

Owner	Task	Location	Time	Photos	Status
Abdul Karim	Fish Feeding	East Pond	08:00-09:00	2	Started
Bashir Ahmed	Vegetable Field Inspection	North Field	10:30-11:30	3	Scheduled
Abdul Karim	Fertilizer Application	South Field	14:00-15:00	2	Scheduled

35. Suggested Main Navigation

Farmer Tab	Purpose
আজকের কাজ	Dashboard and current work
কাজসমূহ	Today, upcoming and history
নোটিফিকেশন	Local simulated alerts
প্রোফাইল	Profile, assigned locations and payroll

Owner Tab	Purpose
ড্যাশবোর্ড	Operational overview
কাজসমূহ	Task planning and recurring jobs
অনুমোদন	Pending Farmer submissions
কৃষকরা	Assigned Farmers and location access
আরও	Farms, locations, incidents, reports and profile

36. Recommended Development Order

1. Create Flutter project, theme, router, Riverpod, mock repositories and data models.
2. Build shared login and role routing.
3. Build Farmer Dashboard and local task list.
4. Build Task Details and task-state handling.
5. Build local GPS/geofence checking with demo-location override.
6. Build Camera → Photo Review → multiple-photo workflow.
7. Build local Submission and late-task detection.
8. Build Farmer history, notifications, incidents, locations, profile and payroll.
9. Build Owner Dashboard and Farmer list/detail.
10. Build Owner location creation and local polygon storage.
11. Build Farmer-location assignment.
12. Build Owner task creation and recurring-task UI.
13. Build Owner approval/rejection workflow.
14. Build incidents and reports.
15. Add role switching and Reset Demo Data.
16. Polish the UI to match the supplied Bashir Agro green-and-white Bangla references.

37. Features Explicitly Deferred Until Backend Phase

- Laravel backend and REST endpoints
- Remote database
- Server authentication/token management
- Firebase push notifications
- SMS OTP
- Cloud image storage
- PostGIS server-side geofence validation

- Real Admin mobile functionality
- Real payroll transactions
- PDF/Excel exports
- Production background scheduler

38. Repository Abstraction - Future-Proofing the Prototype

The most important technical decision is to make the UI depend on repository contracts rather than mock data directly. Example:

```
abstract class TaskRepository {
  Future<List<Task>> getTodayTasks(String farmerId);
  Future<Task> getTask(String id);
  Future<void> startTask(String id);
  Future<void> submitTask(String id, Submission submission);
}

Prototype: MockTaskRepository
Later: ApiTaskRepository
```

Migration Goal

When Laravel is added later, screens, navigation and most business-flow code should remain intact. Only repository implementations and authentication/network services should change.

39. Copy-Paste AI Coding Agent Brief

AI Agent Instruction

Build an Android Flutter prototype named Bashir Agro with two mobile roles: Owner and Farmer. There is no API or backend in the prototype; all users, farms, polygons, assignments, tasks, submissions, notifications, incidents and payroll data must come from local mock repositories and should optionally persist through Hive. Login is simulated using predefined phone/password accounts and routes the user according to role. A Farmer can work for multiple Owners simultaneously and sees all tasks from every Owner combined on one dashboard. Every task belongs to exactly one Farmer and one registered geofenced location. Farmers open a task, view Owner/farm/location/time/instructions, start work only after location verification, then capture one to five live photos. Gallery upload must not be used for proof. Each photo must independently store capture time, latitude, longitude, GPS accuracy and whether it is inside the location polygon. An outside-geofence photo may be captured but must be marked invalid and cannot count toward submission. Late submissions remain allowed but show a red late flag. Submitting creates a local Submission and changes the task to pending review. The Owner sees pending submissions, reviews the photos and location details, and approves or rejects within a displayed 24-hour review window. Rejection requires a reason and allows the Farmer to create a new submission without deleting previous attempts. Owners can create locations, draw polygon boundaries, assign existing globally created Farmers to one or more locations for permanent or temporary periods, create one-time or recurring tasks, view Farmer activity, manage incidents and view reports. Owners cannot see Farmer NID or salary. Farmers can report independent incidents with location and live photo; the incident appears in the Owner interface. Include local notifications generated by actions, demo-user switching, GPS simulation for inside/outside geofence testing and a Reset Demo Data function. Build the app using a repository architecture so mock repositories can later be replaced by Laravel API repositories without changing the UI or business-flow code. The design should closely follow the provided Bashir Agro green-and-white Bangla reference screens and remain simple for Farmers while giving Owners more management controls.

40. Final Prototype Success Criteria

- Farmer and Owner can log in with separate demo credentials and are routed correctly.
- Rahim demonstrably works for multiple Owners and sees combined tasks.
- Owner can create a location with a stored polygon and tolerance.
- Owner can assign an existing Farmer to one or more locations.
- Owner can create one-time and recurring task data.
- Farmer can start a task only after valid location simulation/GPS.
- Farmer can capture multiple live photos and each photo stores its own location metadata.
- Outside-geofence photos are visibly invalid and do not satisfy proof requirements.
- Late submission is accepted but clearly flagged.
- Owner can review, approve or reject; rejection preserves previous submission history.
- Farmer can report incidents and Owner can update incident status.
- All actions update local notifications and survive app restarts when persistence is enabled.
- Switch Demo User and Reset Demo Data make the app easy to present repeatedly.
- Architecture allows mock repositories to be replaced by Laravel API repositories later.

BASHIR AGRO - Prototype first, backend second.

This document is the functional source of truth for the first Flutter prototype.